



**UNIVERSITÀ DEGLI STUDI DI ROMA
TOR VERGATA**

FACOLTÀ DI INGEGNERIA

**CORSO DI LAUREA MAGISTRALE IN INGEGNERIA
INFORMATICA**

**A lambda architecture based solution for analytics
[provvisorio]**

A.A 2014/2015

RELATORE

Valeria Cardellini

CANDIDATO

Luzzi Matteo Remo

CORRELATORE

Glenn Skare Pedersenn
Thomas Aalén

Contents

Acknowledgement	1
Abstract	2
1 Introduction	3
1.1 Background	3
1.1.1 Big Data	3
1.2 Introduction to web analytics	4
1.2.1 Web analytics metrics	5
1.2.2 A concrete example: Google Analytics	6
1.3 State of the art for data system	8
1.3.1 CAP theorem limitation	9
1.3.2 Lambda Architecture	10
1.3.3 Alternatives to lambda architecture	16
1.3.4 Architectures comparison	17
2 Big Data tools	18
2.1 Kafka	18
2.2 Hadoop	21
2.2.1 HDFS	22
2.2.2 Oozie	24
2.3 Spark	24

2.4	Storm	26
2.5	Elasticsearch	28
3	Architectural design	30
3.1	Case: Vimond Media Solution	30
3.1.1	Vimond current architecture	30
3.2	Proposed solution	30
3.2.1	Data ingestion	33
3.2.2	Real time layer	33
3.2.3	Batch layer	34
3.2.4	Serving layer	37
3.2.5	Considerations	38
4	Implementation	40
4.1	AWS Deploy	40
4.2	Data ingestion	40
4.3	Output results	42
4.4	Batch layer	44
4.4.1	EventFetcher	44
4.4.2	Batch data pipeline	46
4.5	Real time layer	52
4.6	Serving layer	52
5	Results and Tests	53
6	Conclusions	54

List of Figures	55
Bibliography	55

Acknowledgement

Finally I'm here writing the last pages of my academic career, a period full of satisfactions and delusions that made me the person who I am now. It would have not been the same without the people who influenced me.

I would like to thank first of all my supervisor Valeria Cardellini from university of Rome "Tor Vergata" for following me in this final project and giving me useful suggestions. I worked on this theis project as an intern in Vimond Media Solution company, located in Bergen (Norway). A great thank to Glenn, Vegard and Kine for considering me as a suitable candidate for this research project, to Helge for being such a great presence, more like a father rather than a CEO and to anyone else who has helped me activetly or just spent time for a talk. I don't know if I could have had a better exerience anywhere else.

università, bergen, amici (friends from a lifetime), famiglia

bergen: Andrea A. Andrea C. Solvejg, Dario, Nicole, Daniele, Alessandra, Josie, Camille, Alix, Julia, Elvira, Roberto

amici: Alessandro, Andrea, Nicholas, Francesco, Marta, Manuel, Piergiorgio, Giuseppe

Finally, the big thank goes to my parents, Angelo and Gabriella. None of this could have ever been possible without their constant support. Thanks for being an inspiration in every desition I took.

Abstract

risalto sull'importanza economica dei analytics data

Chapter 1

Introduction

1.1 Background

In this section we will give an overview of the foundation this thesis project is based upon. We will briefly introduce the big data concept and its analytics derivation. We will focus then on what it is the state of the art concerning software architectures for big data collection and analysis giving pros and cons for every analyzed solution.

1.1.1 Big Data

Big data is one of the trending terms in the IT field which increased in popularity from 2012 [1]. A lot of definition has been given to it [2], focusing on different aspects regarding the data themselves and the infrastructure needed to process them.

Despite the recent dramatic increase in popularity, big data is a concept already coined in the early 2000s. L. Douglas first introduced the 3Vs concept regarding the new challenges that systems had to deal with: Volume, Velocity, Variety [3]. The report focused on what are the problematics due to the increasing size of data, their frequency at which they are produced and a wider range of representation formats. Since 2001, when this report has been published, the question about big data hasn't changed a lot. Of course now we have better approaches and technologies, but also the big data concept

changed. We entered in the IoT era, where every device has internet access, from smartphones to electronic furnitures. Hence the massive amount of data needs new technology for processing and analyzing it. A study from McKinsey Global Institute [8] stated that the amount of data will grow by 40% within 2020, and most likely this trend will not end soon. Big steps in technology has been made to handle this ever-growing data amount: for instance, traditional RDBMS have showed their limit in scalability, hence a NoSQL solution is generally preferred since the data load can be spread across several machines. Moreover, we switch from physical servers to the cloud computing, where virtual resources can be requested on demand only when needed, avoiding for a company the burden of investing a lot of money in a static architecture solution.

For what stated above, always new patterns and solutions for big data processing are needed and the aim of this thesis is to investigate a new approach called **lambda architecture** (section 1.3.2) applied to a subgenre of big data processing: big data web analytics.

1.2 Introduction to web analytics

The discipline of web analytics started in the early 1990s, as a consequence of the spreading of public internet [4]. Since then, monitoring users activity has become crucial, specially for business purposes in order to improve efficiency and quality in organizations.

Almost every online business keeps track of their users, not only their actions (for instance: visiting a certain webpage), but also information about devices used for accessing internet, connection speed, browser and so on. Normally these information are then analyzed and used for user behaviour prediction and contents improving, like

designing the user interface for a certain webpage [5]. Clickstream data is the primary source of web analytics, it refers to the data collected from users activity, which are every kind of interactions with the website.

Nevertheless, web analytics scope is not enclosed into what stated above, typical use cases involve also social network analysis, security informatics or artificial intelligence [6] [7]. Despite the use case, web analytics is just the first step in the overall process of data analysis, they need anyway to be understood by analysts for effective decision making.

1.2.1 Web analytics metrics

Define appropriate indicators for web analytics is challenging. Big data became a powerful source of information only when a sense is given to them, hence standard terms and metrics are needed for the process of analyzing them. Digital Analytics Association (formerly Web Analytics Association) was founded in 2004 with the mission to make analytics a professional discipline for the business, providing a set of common terms, definitions and best practices. DAA defined in a document [11] from 2008 the classification of web metrics in two categories:

- *count*: the most basic unit of measure. An example is the number of visitors for a web page
- *ratio*: a derived metric, obtained dividing a metric by another one. An example is the number of page views per session

A web metric can be *aggregate*, *segmented* or *individual*. An aggregate metric is applied to the totality of the traffic for a website, a segmented one only to a subset of the traffic, and an individual one refers to the traffic generated by a single user. Examples of it

is the average session time for the totality of the users, for the ones connected from a particular nation and for a single user.

Even though DAA tried to standardize web analytics, there is still a certain degree of subjectivity when defining web metrics. A clear example is the concept of user session: for DAA a session ends after a logout event or a timeout of 30 minutes. Google Analytics ¹ instead, define a session a sequence of user actions valid till one of the following event occurs:

- the user stays idle for 30 min.
- day-change at midnight
- the user change the adv. campaigning for accessing the website.

1.2.2 A concrete example: Google Analytics

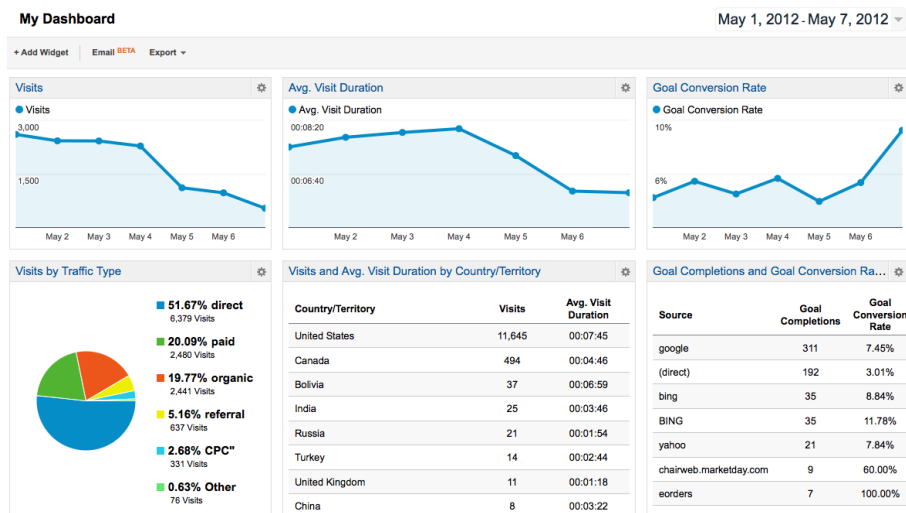
Google launched its analytics service in 2005 and released a free to use version one year later. Soon it became one of the most used tools for website data analytics.

GA tracks the user behaviour through a combination of a javascript library called *analytics.js* and cookies set on user browser for its identification. What made this tool so famous is probably its usability: no particular IT knowledge is required to set the tool up and running: a website owner can simply set via web console all the metrics needed, included conversions.

By default GA uses a javascript snippet called *google analytics tracking code (GATC)* injected in webpages. As the browser loads a page after an HTTP request, GATC is activated and it requests the *analytics.js* library from Google servers, needed for performing the actual data collection through tracker objects. HTTP requests contain

¹<https://support.google.com/analytics/answer/2731565?hl=it>

²<http://ppchero.wpengine.netdna-cdn.com/wp-content/uploads/2012/05/dashboard-.png>

Figure 1.1: Google Analytics dashboard²

valuable information about the client executing the request itself, such as user agent, hostname, origin and language. In addition *analytics.js* sets and reads *first-party* to keep track of the current user. After the tracking has been complete, data are sent to the GA servers as a list of attached parameters to a single-pixel GIF image request for being processed and analyzed.

There are several issues related to GA which can mine its accuracy. First is the cookie usage: setting a cookie to identify a particular user when data are sent to the server for the analysis. If the user deletes the cookies set by the analytics library, it will be considered as a new user by the server, altering then the analysis (for instance when tracking the number of unique visitors on a web page). Secondly, GA uses sampling, which means that not all the clickstream is gathered and analyzed, but only a sample of it. This can clearly lead to a drop in the accuracy of the final results.

1.3 State of the art for data system

As stated above, dealing with analytics data could be misleading, and normally *key performance indicators* could be interpreted differently. Because of this, a lot of companies prefer to build their own in-house analytics systems rather than use standard ones (like GA). Analytics system are a particular case of Big Data system, then when developing such a system, we can apply the same theoretical background. Distributed systems are normally preferred in place of standalone architectures as they offer better performances due to scalability, but they have also some drawbacks: they are complex and less consistent due to the distributed nature. Currently big data systems fall into these two categories:

- *batch processing* of big volume of data with high latency
- *real-time processing* with short latency (milliseconds) keeping the data in-memory.

Batch solutions have been very popular in the last years, specially after Google's publication about *Map-Reduce* computation paradigm [22]: a set intermediate of key-value pairs is generated after a *map* function, and a final set of key-value pairs is the result of a *reduce* function applied to the intermediate set aggregated by keys. The aim is to merge input values in order to generate a possibly smaller set of values. Map-Reduce provides for an high level of abstraction over classic parallel and distributed computation: a programmer only needs to define a map and a reduce function and as a result the computation is spread across several nodes in a cluster. A classic example of this is counting the occurrences of each word in a document. A map function for this problem takes in input a line of the document and emits a tuple for each word in the line. The corresponding reduce function instead just counts the number of tuple received for a key (tuples are distributed across the reducer task by key). There exist several

opensource solutions inspired to Map-Reduce, the most famous one is Hadoop³.

Real-time computation has been introduced to overcome to the high latency which characterizes the execution of a batch process. Data are considered as a stream and the goal is to process them as fast as possible. This approach is often coupled with business intelligence and data mining processes. An interesting overview [10] defines real-time processing system as a evolution of traditional DBMSs. Even if there are substantial differences, as for instance that data needs to be stored before being processed, both systems process incoming data with a sequence of transformation based on common SQL operation defined by relational algebra like *joins*, *filters* or *selections*. A third minor class of data processing handles big data *in-memory*. A study from 2012 [9] predict the in-memory processing will soon overcome the classical batch approach. It highlights the benefits of having a fast data processing system, specially when it comes to react fast to new data with decision making processes. Of course one of the drawback of this approach is the cost: memory is 10 times more expensive that disks, hence companies should go for such a system only when really needed. Another possible issue is related to the dimension of the data to be analyzed: a dataset could be too big to be kept in memory. In these situations, a batch approach is preferred.

1.3.1 CAP theorem limitation

CAP (Consistency-Availability-Partition) theorem states that, in every system, only two of the three previous features can be achieved. In presence of distributed systems we are forced to have network partitions, then the CAP theorem says that we could only choose consistency over availability or viceversa. This means that we can either have a system capable of always serving a user request sacrificing the consistency of the

³<http://hadoop.apache.org>

data, or a system that sometime isn't available, but gives out consistet data (between all the partitions). Keeping this in mind, we will explore the state of the art for the class of system architectures which aims to mitigate the effects of this theorem.

1.3.2 Lambda Architecture

Data system are particularlly influenced by CAP theorem, so a different approach is needed in order to find a tradeoff between availability and consistency. As stated before, a system can perform batch or (nearly) real-time operations on its data. We will explain in the next sections how lambda architecture aims to find a balance between them.

Motivations and architecture

Lambda architecture is a architectural pattern for big data processing system proposed by Nathan Marz [13]. It aims to address to the problem of quering a data system giving result *as much as possible synchronized* with the input data. This imply an user to be able to query the system in real-time in order to obtain information either on what's happening now, or what happened one day ago or also one month ago.

According to its mainifesto [13], this type of architecture has to fullfil the following requirements:

- robusness and fault tolerance agains hardware failuers and human mistakes
- low latency of read and updates operations
- scalability
- generalization in term of different use cases covered by such a system
- extensibility i.e. the possibiliy of adding new features easily

- ad-hoc queries support

The starting point is the simple equation $query = function(all\ data)$ [13], which means that a data system simply answers questions about a dataset, no matter how complex is the query. The problem is that, when dealing with big data (gigabyte/petabytes), a query would take a considerable amount of time to be executed: in this case the result could not be consistent with the entire dataset, since while computing it, new data might have been injected into the system. Unfortunately there is no such a single system capable to answer to every kind of query *in real-time*.

The first concept introduced by lambda architecture, is to precompute query functions [15] in order to be able to answer quickly to them. If an user now, queried a particular timeframe of data, the system would answer immediately by showing the precomputed result. Computing aggregated results imposes high latency, a typical instance of aggregation deals with data collected in the last hours or even days and it may take hours to complete, hence the last step is how to fill the gap between the aggregated results and the new data not covered by aggregations.

In order to solve this problem, lambda architecture proposes a three layers architecture composed by:

- Batch layer
- Real-time layer
- Serving layer

Every single data ingested into the system is replicated into and elaborated by either the batch and real-time layer in order to have intermediate results (we will refer to them as *batch* and *real-time views*). The outcomes are then merged into the serving

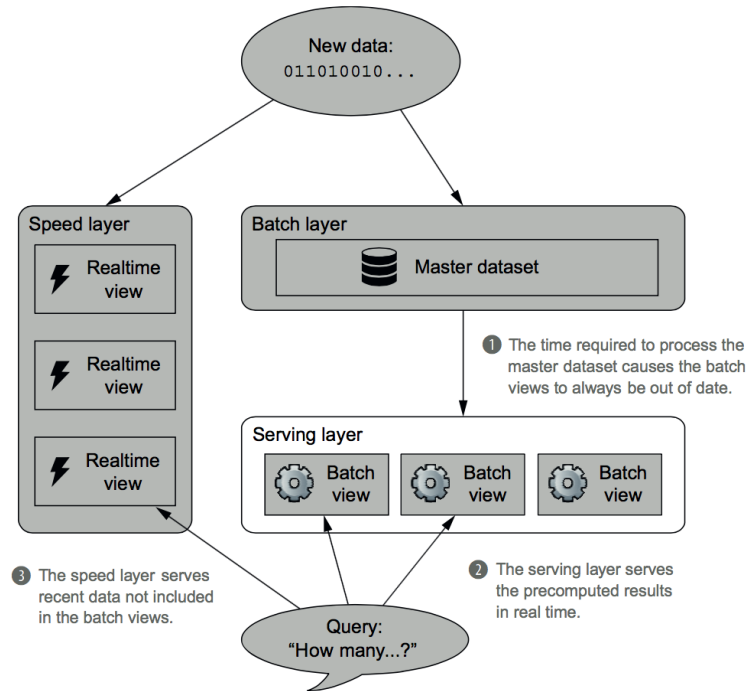


Figure 1.2: Lambda architecture as union of three layers[15]

layer and ready to be queried by end users. Each layer has different characteristics according to the role they fulfill in the overall architecture.

The batch layer is composed by two parts: a master dataset and a batch computation part. The master dataset holds the data coming into the system needed for calculating the aggregated results. Data are considered immutable, no updates or deletion are *in theory* allowed: data can be appended into or read from the master dataset. We will show later how this is a great advantage in terms of consistency. The batch computation part reads from the master dataset and computes batch views. According to the equation proposed by Marz [15], the batch layer would solve the equation $batch\ view = function(all\ data)$. For the latter operation two approaches are possible: incremental and recomputation. The first approach implies an update of previous views in order to compute the current one, the second instead involves a total recomputation

based only on the raw data stored in the master dataset. Recomputation algorithms are easier as they result in a simpler logic, while sometimes using incremental algorithms may speed up the whole process as it exploits the existence of already computed results. The choice of the best approach really depends to the use cases the system has to address to. For instance, let's assume we want to calculate the total number of visit for a webpage over the time. It is easy to understand that for this use case an incremental approach is preferred over a total recomputation on the entire dataset executed everytime new data come. The difference in time tends to diverge according to the quantity of data the batch is computed on.

The real-time layer aims to fill the gap introduced by the high latency of the batch layer and it is responsible for providing results on the most recent data. It takes the same input of the batch layer, but instead of storing it in a dataset, elaborates it on-the-fly. It behaves as the batch layer, in terms of precomputing the same type of views, the difference is that it works only with a small portion of data, exactly the one which it is not covered by batch views. An incremental approach is generally preferred and real-time views are update as new data comes into the system. Formally it solves the equation $realtime\ view = function(realtime\ view, new\ data)$ [15]. Overall the design of the real-time layer is more complex of the batch layer, either for its incremental approach, either for the requirements it has to respect such as short latency. Fortunately it is only responsible for a short dataset and its views are eventually corrected by the batch ones, so normally a certain degree of approximation is here tollerate. We will go more in detail into this in the next section.

The serving layer is responsible for providing results to ad-hoc queries with low latency as a combination of batch and real-time views. It indexes results from the batch layer in order to serve them with low latency and it compesate the lack of batch views for the

new data by querying the real-time ones. Hence it gives a final answer to the problem of querying a data system by solving the equation $query = function(batch\ view, real-time\ view)$ [15]. It is the layer with the most complex logic as it has to merge different views according to type of query submitted to the system. A simple case is when the data are time ordered and a particular query refers to a fixed timeframe: the serving layer has just to aggregate batch views with, if needed, real-time ones. Though this is just a possible use case and a general purpose implementation of the serving layer might be not trivial.

CAP theorem & LA

Even though the audacious title of Marz article about lambda architecture (*i.d.* "How to beat CAP theorem" [13], CAP theorem does still hold [16]. What lambda architecture aims to achieve is the so-called eventual consistency. This means, that, after a certain timeframe, data are eventually consistent across all the partitions of a distributed system.

We stated before that every single layer of the lambda architecture has to respect different constraints due to their different role in the overall system. The master dataset has only to support creation and read, making then its data an immutable source. This is a great advantage, in the sense that the system does not have to deal with updates, which are one of the causes of lack of consistency in distributed systems. The overall batch layer is eventually consistent in the sense new data are consistent with the query results only after a batch view has been calculated. Moreover, a database supporting random reads and batch writes is normally used for storing such these views. The real-time layer instead, has to be highly available in order to compensate the latency

introduce by the batch layer. In order to compute real-time views fast, often are used approximation algorithms rather than exact ones (for instance the evaluation of distinct records on set is often estimated using *hyperloglog* algorithm [17]) and the database used to store the real-time views has to support either random writes and reads due to the incremental approach used by this computation.

The overall lambda architecture based system, as we said, is eventually consistent. Approximations and mistakes made by the real-time layer are eventually corrected by the batch views, once they are computed.

Pros and Cons

One of the main problems lambda architecture wanted to solve is the ability of querying data in real-time. In a certain way this has been achieved: high latency introduced by batch computation is compensated by real-time views and at every time the system is able to respond to a query, with a certain degree of accuracy as previously stated. This is indeed a great result, specially compared to what we can achieve with regular data system (real-time computation **or** batch computation).

Having an immutable source of data gives to the system robustness against failover : if an error happens on the batch layer, either an error in the processing logic or an hardware failure, the entire computation can be run over again without any data loss. Calculation of the final result as a series of transformations of an immutable dataset is a concept also used in batch/micro batch computation framework like Spark [19] to achieve *exactly once* message semantic.

The main weakness of such a system is without doubts its complexity. The serving layer is the one with the most complex logic as we explained before, specially in case of query not dependent only by a time frame. Also, under a developing point of view,

at the moment it does not exist a unified framework in order to share the same logic between different layers. An attempt of unification has been made with twitter summingbird project [18]: it aims to provide for an high level abstraction for defining the logic of a job and then share it among different framework for batch and real-time computation. Nevertheless this is still a raw project rather than a production-ready solution. Another drawbacks is the lack of flexibility regarding the queries such a system can serve. One of the requisite of lambda architecture is the ability to handle *ad-hoc* queries, in the sense they depend on parameters known only at query-time. What lambda architecture lacks instead, is the ability to serve arbitrary queries since it deals only with precomputed views. A simple workaround would be to define a fine timeframe granularity (for instance 15 minutes) and then compute and store bigger views (in term of time) as combination of the thinner ones.

1.3.3 Alternatives to lambda architecture

Lambda architecture isn't perfect, and other solutions found in literature and in proof of concept ideas try to exploit the weakness of it. We are going to illustrate alternatives..

Kappa Architecture

The main competitor of Lambda architecture is an architectural pattern called *Kappa architecture*[20] first mentioned by Jay Kreps (project leader of Kafka at LinkedIn). This approach claims to beat Lambda architecture in its main weakness: the complexity of the overall system. Kreps says that maintain such a complex system likely lead to lack of synchronization between the real-time and the batch layer as well as an high effort for running and debugging it.

The approach proposed with Kappa architecture is to exploit just the real-time computation layer working with a system able to persist messages over the time. In lambda

architecture, real-time and batch layer share the same logic even though they deal with different data fragment. In this approach data are stored and reprocessed several time by the same stream processing logic according to the different timeviews required by the system. Each processed result is then stored in a serving layer in change to switch between them according to the queris executed by end users. In this scenario you might have parallel processing processes working on the same data in different point: a strem could be processing data from one month ago, one from one week ago and another one could just follow the real-time flow of new data.

Real Time Streaming architecture

iot architecture

1.3.4 Architectures comparison

Chapter 2

Big Data tools

In this chapter we will describe the tools used for implementing a working prototype based on the lambda architecture paradigm previously explained. The scope of this thesis is not to make a survey of modern technologies used for big data and here we will give just an overview highlighting the purpose and the main features of each used tool. In the appendix the basic configurations for each tool are showed. For a better understanding of these tools, the reader is invited to consult additional resources about them.

2.1 Kafka

Apache Kafka [21] is a general purpose distributed publish-subscribe message system originally developed at LinkedIn and then open sourced in 2011 and inserted in the Apache ecosystem. The architecture is composed by one or more kafka servers, also known as *brokers*, which rely on Apache Zookeeper for coordinations.

The key abstraction in Kafka is a topic: *producers* publish message to a topic and *consumers* subscribe to one or more topics. A single topic is composed by a set of partitions, distributed and replicated among all the brokers in the cluster. For each partition there is a master replica and a set of slaves. Since normally there are more

partitions than brokers, a broker can play the role of master some partitions and the role of slave for others. On each partition are stored metadata about the position of consumers that subscribed to that topic, also called offsets. An offset represents a checkpoint for that consumer, messages within that are assumed to be correctly processed and the next time the consumer makes a fetch request, the broker will forward messages starting from that offset. The consumer, anyway, is still able to reprocess already committed messages, for instance in response to a failover event.

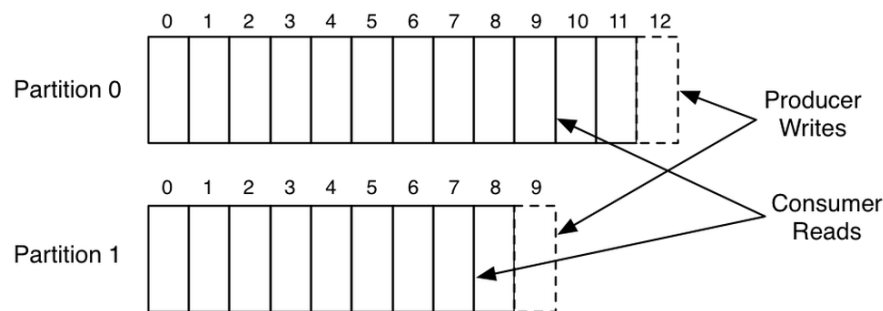


Figure 2.1: A 2-partitions topic with one producer and one consumer.¹

What it is interesting in Kafka is how messages are forwarded from the producer to the consumer through the brokers. Unlike other message systems which process the messages in-memory, Kafka does a strong usage of the filesystem: messages are immediately stored on filesystem and not deleted when read by a consumer but retained according to a configurable period. This avoids any kind of data loss for every committed message, even in case of brokers failure. The only information retained by the broker is the position of the consumer for that topic (which are the offsets for the partitions the topic is composed by).

¹<https://engineering.linkedin.com/kafka/benchmarking-apache-kafka-2-million-writes-second-three-cheap-machines>

Kafka allows to implement either a *queuing* or a *publish-subscribe* logic introducing the abstraction of **consumer group**. Each consumer labels itself with a group name and is responsible for one or more partitions of a topic. Kafka ensures that no more than one consumer per group reads from a partition: if in the group there are more consumers than partitions, some of them will stay idle. In the opposite scenario, a consumer will read from more than one partition. If all the consumers belong to the same group, then the system works as a queue, balancing the load across the consumer group. If all the consumers have different groups, then the system works as a standard publish-subscribe.

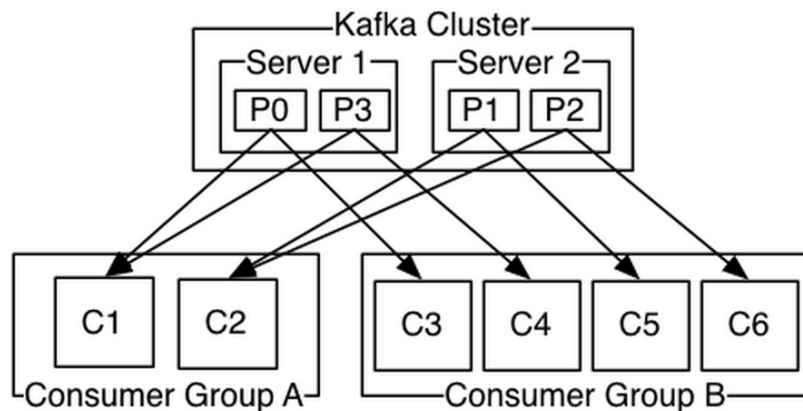


Figure 2.2: Two consumer groups read from a 4-partition topic.²

Having the concept of the consumer group implies a total order over messages within each partition: the order on how are messages are read inside a partition is the same for every consumer who subscribed that partition.

As a conclusion, is worthy mention that Kafka implements a *at-least-once* message semantic in the following way:

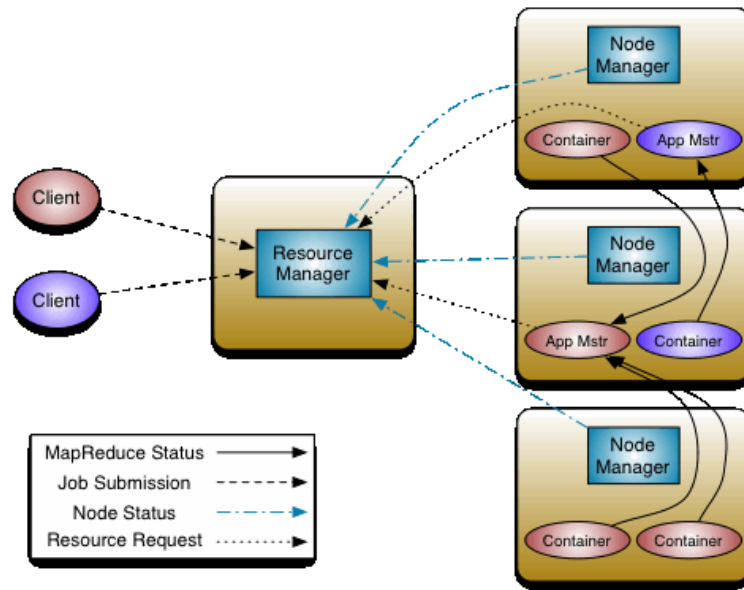
²<http://kafka.apache.org/documentation.html>

- Producer: when publishing a message and having a network error, the producer can't be sure if the message has been correctly committed to the broker.
- Consumer: a message is said to be fully processed (from a broker perspective) only when its offset has been committed. The best the consumer can do is, read the message, process it and then commit the offset. If the consumer crashes before committing the offset but after processing the message, the one taking over will process the same message.

2.2 Hadoop

Hadoop is an open-source suite for scalable fault-tolerant distributed computing written in Java. Based on Google's *Map-Reduce* [22] approach, it was first developed in order to run MapReduce jobs for processing a web crawl. The rapid increase of popularity led to an abuse of the original computing paradigm exposed by Hadoop such as run heterogeneous task in a multi-tenant environment. In 2013, a second version of Hadoop, YARN [23], was released with the purpose of compensate the limitations of Hadoop first version, in primis decouple the programming model from the resource management and support a multi-tenancy job scheduler. The architecture of YARN is composed by a global per-cluster *Resource Manager (RM)* with duties of scheduling jobs submitted by clients and allocating resources when needed by the applications running on the cluster. Having a centralized coordinator enables scheduling properties of fairness, capacity and locality. The architecture is completed by one or more *Node Managers (NM)* running on each slave node in the cluster. NMs are the workers daemons, in charge of monitoring the execution of the applications running on its node. Resources are spawned across the application in form of *containers*. A per-application daemon

³<http://hadoop.apache.org/docs/r2.7.0/hadoop-yarn/hadoop-yarn-site/YARN.html>

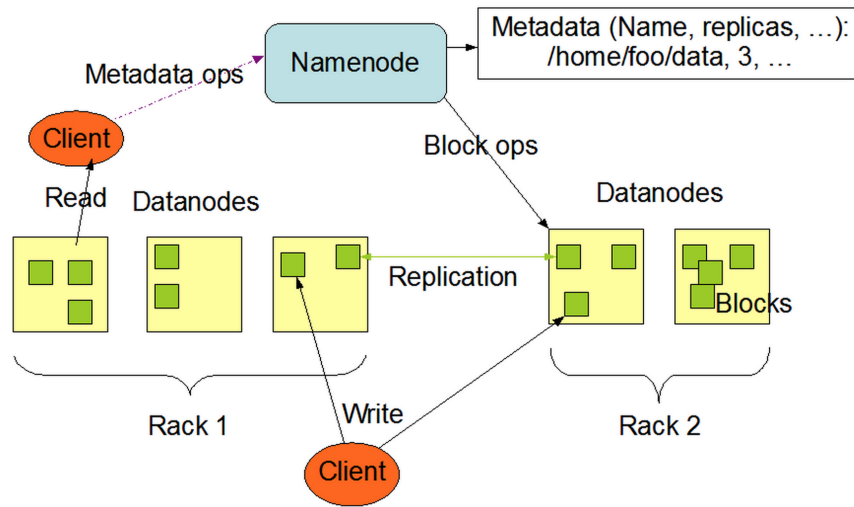
Figure 2.3: YARN architecture³

called *Application Master (AM)* dynamically requests to the RM bundles of resources (containers) in order to execute the application it is responsible of. In this scenario, the RM is agnostic of the applications logics as it only replies to resources requests. MapReduce is now only a class of applications Hadoop YARN supports and several computation framework implementing different programming models has been adapted for being executed on top of YARN (Spark, Storm, Tez, Giraph and others).

2.2.1 HDFS

Hadoop distributed storage implementation is called HDFS (Hadoop Distributed File System). It is an highly efficient storage solution perfectly integrated with applications running on Hadoop supporting the main operations of a normal filesystem and structured in a folder hierarchy. It is composed by a master/slave architecture with by one *Namenode* and at least one *Datanode*. A Namenode holds the namespace of the

⁴<http://hadoop.apache.org/docs/r2.7.0/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>

Figure 2.4: HDFS architecture⁴

filesystem, i.e. information about the structure of the filesystem and metadata about the distribution of the files over the datanodes, and listens for client requests such as writes or reads. Datanodes are the worker nodes: they store data and they serve the actual client requests, providing for data in case of read requests or allocating space in case of write requests. In this architecture, Namenode represents a single point of failure, the namespace is kept in memory by the Namenode and in case of its failure all data is compromised. There are several solutions to run HDFS in *High Availability (HA)* mode such as maintaining a *StandBy* replica synchronized with the master through a network file system [24]

A file in HDFS is split in several blocks according to its size (HDFS handles blocks of 64Mb by default) and distributed and replicated across the Datanodes according to a per-file policy. The position of each block and their replicas are kept in the Namenode's namespace. Having a block abstraction gives robustness and fail-tolerance to the storage system in case of Datanode failure: client requests are normally redirected to the nearest healthy Datanode containing the required information.

2.2.2 Oozie

Apache oozie[25] is a workflow scheduler for Hadoop. Each workflow is composed by a sequence of action executed in sequential mode, which means that an action can't be executed until the previous one ended. If configured with YARN resource negotiator, Oozie runs each job in a container context [2.2]. Oozie workflow is build on a DAG (direct acyclic graph) stucture and is composed by a sequence of two possible type of *nodes*:

- control flow: nodes that control the start, the end and the execution of the workflow without executing any concrete action. Example of this nodes are: start control node, end control node, kill control node, fork control node.
- action: nodes that execute a task. The actions supported by the latest version (4.2.0 July 2015) are Map-Reduce, Pig, Java, Fs (filesystem), ssh, Spark

Jobs are scheduled by the server with a minimal granularity of minutes. It is possible to specify an input and an output directory where to read data needed to the job and where to store an eventual result. Moreover, the input and the output directories path can be expressed in a dynamic way using time expressions which are evaluate differently according to when the job is executed. All the configuration is submitted to the server using an XML file containing information about the position of the application executables, directories paths, application arguments and frequency of the jobs.

2.3 Spark

Apache Spark is a distributed framework for data computing. It is based on the same paradigm of Hadoop/Map-Reduce, where the computation is split across several nodes in the cluster (*map*) and the final result is given by aggregating the intermediate ones

(*reduce*). Spark has been designed for the class of applications which need to iterate several time on the same dataset, such as machine learning and interactive analytics, allowing them to store intermediate data in memory instead of recompute it at every step. For these type of application, it has been showed how spark outperforms a classic map-reduce paradigm by 10 times [26].

In order to be executed in a distributed mode, Spark needs a cluster manager for resources allocation across all the nodes. The architecture is composed by a primary node called *driver program* and a set of worker nodes running *executor* threads. When an application is submitted to the cluster, the driver program spreads it across the worker nodes and schedules task to be run by each executor in a isolate context (each executor runs its own JVM). In this way there is complete isolation between several applications running on the same cluster, the drawback is that executors belonging to different applications can't share data between themselves and also within the same application they need the support of the driver program for sharing data using special objects such as *broadcast* and *accumulator* variables.

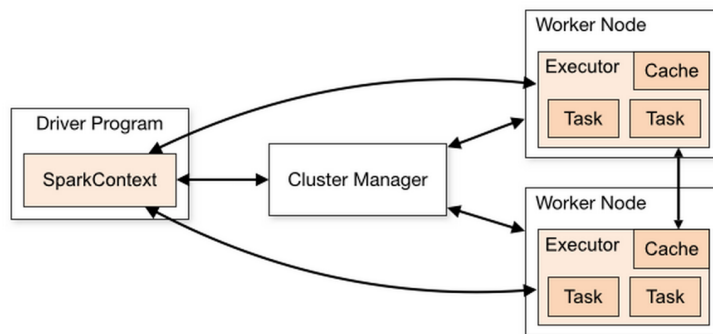


Figure 2.5: Spark cluster architecture⁵

The main concept around spark is an abstraction over a physical dataset called *RDD*

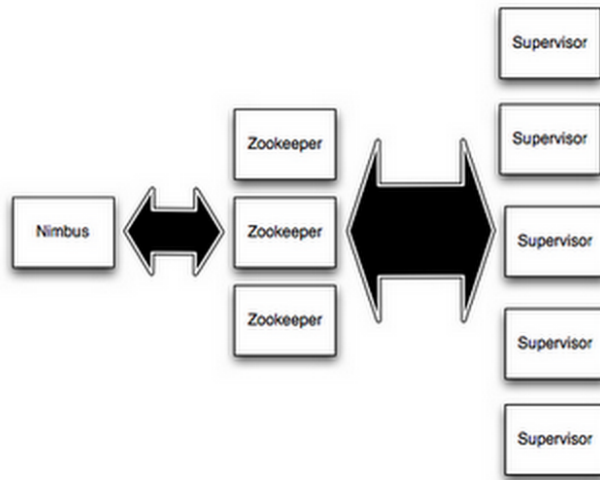
⁵<https://spark.apache.org/docs/latest/cluster-overview.html>

[27] (resilient distributed dataset). RDDs are read-only, fault-tolerant parallel data structures divided into one or more partition across a set of machines. They can be created from physical files, parallelizing a collection of elements or transforming an existing RDD by applying a transformation (ex. map, filter...) or an action (ex reduce, count...). RDD are lazy by nature, in the sense that the driver program keeps in memory the list of transformations needed to compute it rather than execute them at each step. Only when an action requires the RDD to be returned to the driver program, the list of transformations is applied. By default, each transformed RDD is re-calculated each time an action runs on it. It is possible however, to cache it in memory for faster successive accesses. This abstraction increases the robustness of the data model: in case of a partition failure, it can be reconstructed by applying the list of transformations and actions from the last RDD available in memory.

2.4 Storm

Apache Storm is a distributed real-time data processing framework created by Nathan Marz (the proposer of lambda architecture). Storm considers the data as a stream of *Tuple*, generated in special component called *spouts* and processed by one or more *bolt*. Spouts and bolt can be combined in a direct graph structure called *topology*.

A Storm cluster is composed by a master node and one or more worker nodes. The master node runs a process called *Nimbus*: it is where the application are submitted to the cluster and it is responsible for distributing the code across the nodes, assign tasks and check for failures. The worker nodes run one or more processes called *supervisors* in charge of communication with the master and of executing worker processes. Each worker process runs different parts of the topology in *tasks*. The master node communicates with the workers through *ZooKeeper*. Each component (spout or bolt)

Figure 2.6: Storm cluster architecture⁶

can be replicated for load distribution. Within a worker process it is possible to run one or more *executor* threads, each *executor* finally runs one or more tasks, which are just parts of the topology belonging to the same component. As stated before, the main abstraction in Storm is the stream of Tuple. Data flows from a component to the next one according to a partition policy: data can be distributed in shuffle mode, by a key, toward a unique component or replicated. Each mode fulfills a different use case: for instance a key-partition (*fields grouping*) is useful when we want to count the occurrence of a particular element in the data stream while a shuffle partition (*shuffle grouping*) is a better approach when the same operation has to be performed on each tuple, independently of the actual content of the tuple itself.

Storm guarantees by default a *at-least-once* message processing semantic. A topology can be launched with acker components, responsible of ensuring that a tuple is successfully processed within a timeout. Storm keeps in memory a DAG for each tuple generated by a spout: each node in the DAG represents a component which is supposed

⁶<https://storm.apache.org/documentation/Tutorial.html>

to process the tuple and sends an ack on success. If the acker receives all the acks for a tuple then that tuple is said to be fully processed, otherwise the tuple is considered failed and the spout responsible for it should be able to regenerate it. In this way no data loss is guaranteed, but a tuple might be processed more than once.

2.5 Elasticsearch

Elasticsearch is a distributed full-text search engine built on top of Apache Lucene. It provides a set of Rest API for documents indexing, querying or real-time analytics. In order to give a better understanding of how documents are organized inside the cluster, it is possible to compare Elasticsearch structure to a relational database where:

- a database is called *index*
- a table is called *type*
- a table row is a *document* in JSON format
- a table column is a *field*

Elasticsearch uses Lucene's inverted index datastructure for fast full-text searches. Basically an inverted index stores, for every unique word inside every document, a list of documents that contains that particular word. Furthermore, each word can be normalized according to a text analyzer: for instance each word can be converted in lower case, or in a singular form, a verb converted to the infinitive tense, or words can be aggregated if they are synonyms. This step is crucial for having a fast search engine and the user can tune it according to its use cases.

Elasticsearch provides a simple master/slaves cluster architecture: either a master or a slave node is capable of indexing or querying documents, the only difference is that

a master node is responsible for coordinating operations such as creating an index or adding/deleting a node in the cluster.

It is worthy mention the meaning of the index as an abstraction for one or more physical dataspace called *shards*. At the index creation time, the user can specify in how many shards to divide an index, and how many replicas for a single shards. This is an important setting since the number of shards can't be changes as long as the index is active, only the number of replicas is dynamic. Every time a node leaves or joins the cluster, the master node balances the shards and the respective replicas over the actual cluster composition.

Chapter 3

Architectural design

3.1 Case: Vimond Media Solution

Vimond Media Solution [12] is an OTT (Over-the-top) services provider company headquartered in Bergen, Norway. Having customers spreaded around the world, Vimond is one of the leader in the streaming media market.

3.1.1 Vimond current architecture

3.2 Proposed solution

The proposed solution is meant to be a proof of concept of lambda architecture, rather than an effective production-ready system. It aims to improve the theoretical idea by simplifying it where there are the majority of problems (i.e. the complexity), trying to keep at the same time the two layer pattern and highlighting what can or can't be achieve with these simplifications. For the rest of the report we will refer to this model as VimondAnalytics.

The main simplification comes when dealing with intermediate views. According to the original lambda architecture structure, each processing layer has its intermediate views to be merged into global ones in the serving layer. The model we are going to propose avoid storing intermediate views in different databases and all data are ingested in

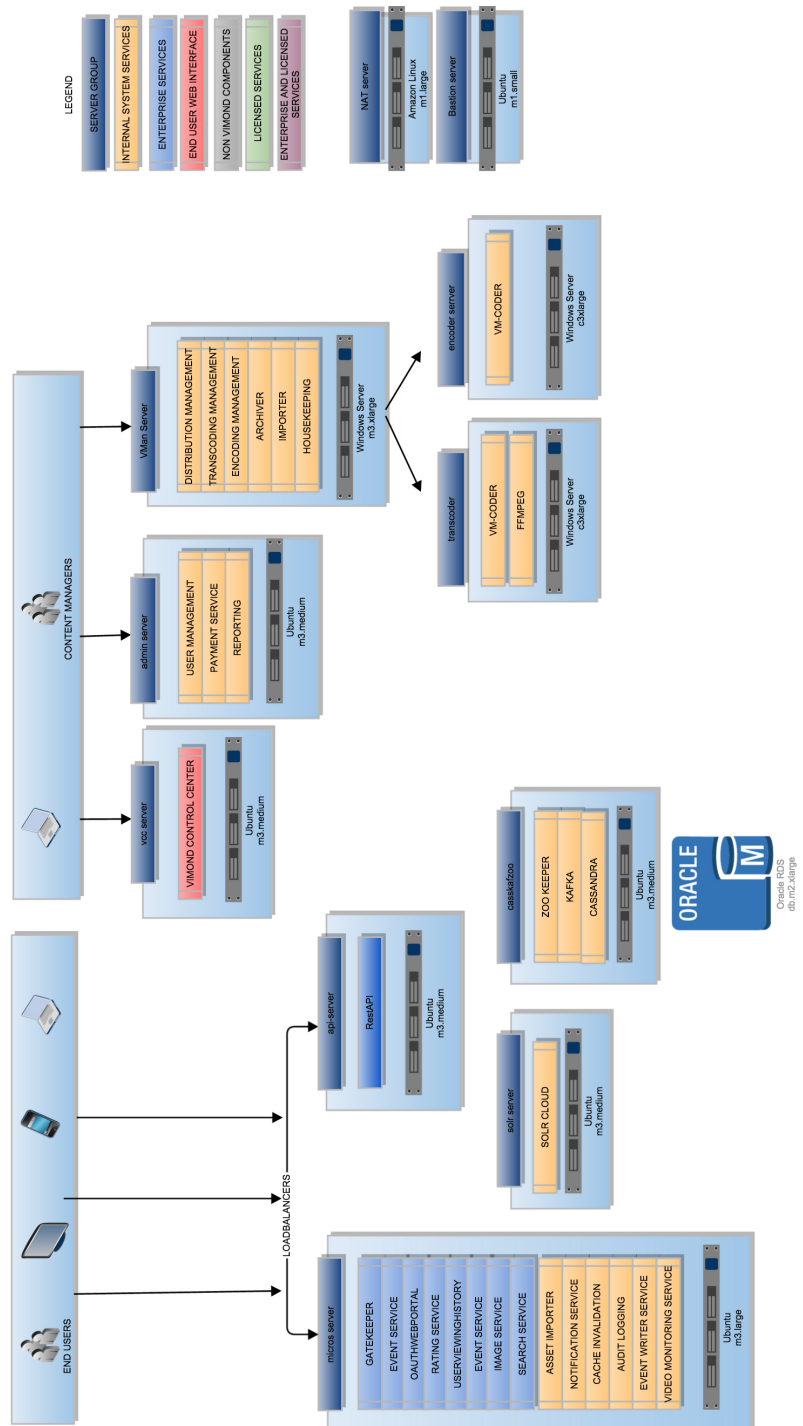


Figure 3.1: Example of Vimond production environment.

the serving layer. The purpose of the latter is similar to the original one as it has to merge data coming from the two different processing layers, but now instead of merging views, it merges raw data coming from the real-layer and aggregated ones from the batch layer. The simplification is hence obvious: there is no need anymore for different storage solutions which have different requirements and then different problems. All the complexity is moved to the serving layer while the real-time and the batch layer fulfill only processing requirements.

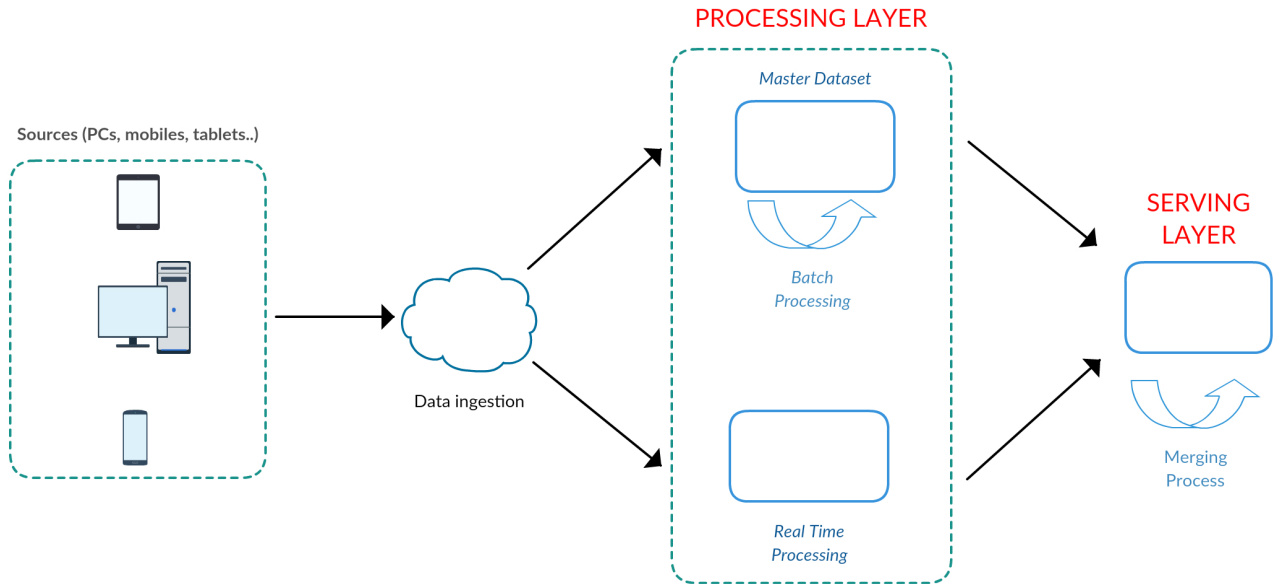


Figure 3.2: VimondAnalytics architecture

The final architecture is then a composition of several layers. Data come into the system through a *data ingestion* layer, implemented with a publish/subscribe architecture where consumers process new messages as soon as they are available. The *real-time layer* gets data from the data ingestion and after few processing steps, store them in the serving layer. The *batch layer* store data in a data lake and runs aggregation processes for obtaining a compact (and shorter) view out of them which will be stored as

well in the serving layer. The latter aggregates raw data and merges them with the batch views in order to answer to end users queries.

The rest of the chapter describes each separate layer of the model above briefly introduced.

3.2.1 Data ingestion

Data ingestion layer represents the entry point to the system, where input data are managed and replicate into the batch and the real-time layer. Apache Kafka fits perfectly into this role, as is it possible to handle different consumers and have them fetching messages from different points on the same dataset, according to the capability of each layer.

In order to get the whole system working we needed a data source. Normally artificial dataset can be used for a proof of concept validation but we preferred to use real ones. Vimond provides TV2 Norway with services for its online streaming platform called *TV2 Sumo*. Data used in this project come from that platform and refer to a player playback functionality which allows an user to resume the stream of an asset when it closes the browser window and wants to re-play from the previous point.

3.2.2 Real time layer

The real-time layer of this model is very simple, its purpose is to forward data from the data ingestion part to the serving layer. Differently from the original approach, it does not have to generate any aggregation or storing data on an intermediate database. A possible process executed in this layer could be data augmentation, like IP translation into location and coordinates or user agent breaking into browser and operative system

version. Neither an incremental nor a recomputation algorithm is used: augmented data are just written on the serving layer. A typical example of data forwarded by the real time layer could be:

```
1  {  
2      "eventType": "click"  
3      "userId" : "12345",  
4      "targetURL" : "http://www.mywebsite.com/mypage.html"  
5      "timestamp": "2015-09-06-19:54:00"  
6  }
```

Listing 1: RealTime data example

Apache Storm has been used for implementing this layer as it provides the features we need. No message guarantee policy is required, we are not interested in having a strong consistency level, data need to be forwarded as fast as possible and a certain degree of inaccuracy is tolerated. Anyway, modern real-time processing frameworks such as Storm allow to implement an *at-least-once* message processing instead of *at-most-once* where messages lost is possible. According to the use cases of the current scenario, it is possible to switch to the policy needed.

3.2.3 Batch layer

The batch layer is organized in two part. First data are fetched from the data ingestion layer and saved into a data lake. Then the data are analyzed and aggregated according to the output views.

In order to improve the performances of the fetching process, data are stored in the data lake in batches and two mode of moving data were implemented:

- **unreliable**: the offset for a message is committed after it has been read from the data ingestion layer. This implies that, if the process crashes or fail to write

the message on the data lake, than the message is lost. In addition, if messages are processed in batch, in case of crash of the process, all the batch is lost. This approach implements a *at-most-once* message delivery semantic.

- **reliable:** the offset is processed after the batch of messages is written successfully on the data lake. In case of crash of the process, the one who takes over it is going to read from the last committed offset which represent the latest checkpoint. In case of crash of the process after writing the messages but before committing the offset, data will be written twice. This approach implements a *at-least-once* message delivery semantic.

Data lake is implemented through Hadoop HDFS. As explained in the previous chapter, HDFS implements an abstraction over a distributed data storage such that classic filesystem operations are possible. Data written into the data lake are organized in a time folders structure manner: the dataset is a hierarchy of folders where at the top there are the ones representing days, the second level is for the hours of a particular day and the third is for the division of the hour in intervals. It is possible to divide an hour or a day in several intervals but partitions bigger than days were not implemented. When a new datum needs to be written in the dataset, it goes to the corresponding folder according to its timestamp. For instance, let assume a partition criteria of 15 minutes: this means that every folder representing an hour contains 4 sub-folders representing the minutes intervals 00-14, 15-29, 30-44, 45-59. The figure 3.3 explains where data are actually written in this scenario, according to their timestamp.

While new data are ingested into the data lake, the batch layer is responsible to execute aggregation processes on the historical ones. Each of these processes takes in input data contained inside a specific folder, which changes over the different executions, and

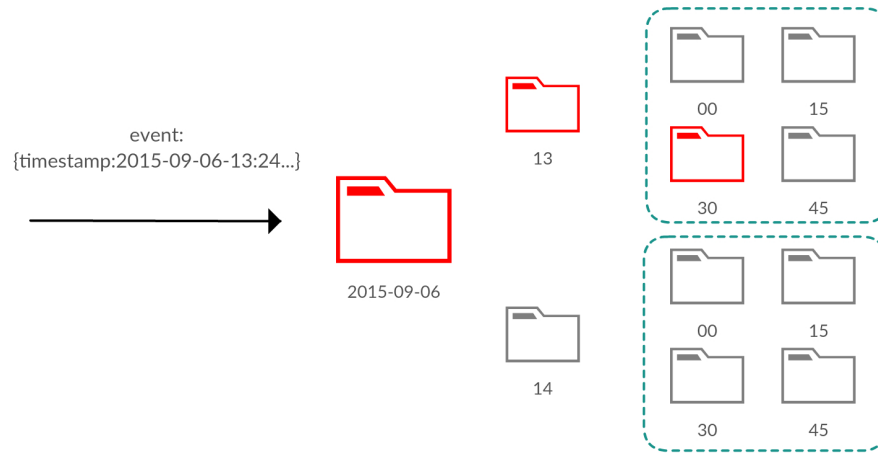


Figure 3.3: Time folder structure: an event is saved in the corresponding folder, according to its timestamp.

produces as output a set of events representing parts of the batch views and they are stored directly into the serving layer, without the use of an intermediate database. A typical example of a data generated by this layer could be: A data coming from the batch layer could be like:

```
1  {
2    "eventType": "click"
3    "targetURL" : "http://www.mywebsite.com/mypage.html"
4    "counter": 100
5    "timestamp": "2015-09-06-19:54:00"
6  }
```

Listing 2: Batch layer data example

An incremental approach can be used, but this model has been thought for a recomputational one over the different data subsets. The different batch views produced at each execution can be then aggregate with each others in the serving layer. While this approach simplifies a lot the whole aggregation processes, it has limitations in the type of results it can produce as explained in the last subsection of this chapter.

Finally, after a batch window of data has been processed, according to the principles of lambda architecture, real-time data for included in that time window are no longer needed as we have an aggregate version, much smaller and faster to query. It is a duty of this layer ensure that, after the succesful completion of a batch process, the corresponding real-time data are deleted. In this way, all the approximation made by the real-time and serving layer are eventually corrected by the batch views.

3.2.4 Serving layer

The serving layer of VimondAnalytics architecture presents the biggest differences with the original idea proposed by lambda architecture. As said in the previous chapter, the serving layer should index the batch views and merge them with the real-time ones. Here instead, we don't have anymore the concept of intermediate views, but only final ones are presented to end users. For this purpose, it is needed a solution which provides storage and aggregation functionalities.

Two kind of data are written into the serving layer:

- data coming from the real-time layer
- aggregated data coming from the batch layer.

According to the data example provided in the previous sections, a serving view regarding the count of clicks for a particular web page merges all the real-time data for that webpage with the ones generate from the batch layer, according to the timeframe requested by the query. If the serving layer uses JSON syntax as well, then an output result could be the one proposed in figure 3 Elasticsearch was used for implementing this layer. We are not interested in the full-text search engine but we exploited mostly the aggregation functionalities on the documents stored into it.

```
1  {  
2      "eventType": "click"  
3      "targetURL" : "http://www.mywebsite.com/mypage.html"  
4      "counter": 200  
5      "from": "2015-09-06-19:00:00"  
6      "to": "2015-09-06-20:00:00"  
7  }
```

Listing 3: Serving Layer data example

3.2.5 Considerations

In this chapter we presented a lambda architecture based model for big data analytics. The idea behind this model is the simplification of the theoretical one, removing intermediate views and then complexity to the whole system and computing final views in the serving layer, as a combination of raw data and batch results.

Concerning the batch layer, looking at the data in time windows causes limitations in the type of results that can be overall computed. Let assume to have two consecutive batch windows of data, created by a time partitioning structure as explained in the section 3.2.3. The different results calculated on each window are then stored on the serving layer and aggregate with each others if a query wants to extract data from a time interval containing both of them. Hence, we need to ensure that the aggregated results are consistent among all the different batches, which is, given two partitions A and B and an aggregation function f , the following equation $f(A) + f(B) = f(A + B)$ must hold. For all the other cases where it does not hold, an incremental approach is needed. Example of consistent views are generated by unique events over the time, like a purchase or a content rating.

Other limitations of this approach are inherited from the lambda architecture approach: it is not possible to query the system for a time interval shorter than the batch time

window. The smaller the granularity, the greater the accuracy of the query, but compute more frequently a batch job (even if applied on a smaller subset of data) might not be the best option in terms of efficiency.

The applicability of this approach lies on the use cases where a normal query would investigate what happened in a fixed timespan in the past, corresponding to one or more batch windows (for instance, recalling the previous example, the number of clicks for a webpage from 10 am to 12 am, assuming a 30 minutes batch interval), or a real-time overview over all the historical data saved in the serving layer.

Chapter 4

Implementation

In this chapter we are going to show a concrete implementation of the architecture explained previously. For each layer we will provide explanation of the most important features using diagrams and code snippets.

4.1 AWS Deploy

elencare le macchine usate e come sono stati deployati i vari servizi su di esse mettere configurazioni?

4.2 Data ingestion

TV2 runs Sumo production environment into a virtual private cloud on Amazon Web Service (AWS), isolated from the outside network and it is not possible to fetch them directly. A typical cloud production environment keeps the virtual machines inside private networks and routes the network traffic to the external network through NAT instances. A bastion server is normally placed as protection against undesired accesses such that only trusted user can access the private cloud.

Since Sumo platform uses Apache Kafka as a message sharing system between several

services, it has been convenient and easy to create a "data bridge" inside that environment for pushing data inside the VimondAnalytics one.

For this purpose it has been used an already existing Vimond project called "Firehose", which has been modified for this use case. Firehose simply implements a two endpoints system. Data are read from a Kafka topic, validated and forwarded to another Kafka cluster. For security reason, Firehose is installed on a machine inside TV2 VPC: in this way the Kafka cluster source is directly accessible through private IPs. In order to connect it to the VimondAnalytics Kafka cluster, an elastic IP is assigned to the latter.

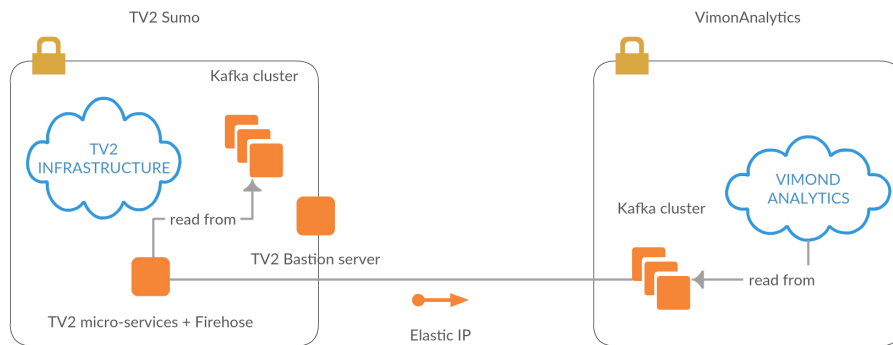


Figure 4.1: Firehose data bridge

Data model

As said in the previous chapter, the data used for this project come from the resume player functionality provided by TV2 pay-per-view platform *Sumo*. Data are generated by players running on different devices: it results in two type of events according to the usage of silverlight plugin. The events generated by other plugins contains a subset of information of the ones generated by silverlight plugin. Specifically, an example of non-silverlight event is:

```
1  {
2      "eventName": "player_log_event"
3      "originator": "player"
4      "timestamp": "2015-09-13T10:31:39.879Z"
5      "guid": "5d33aa50-8232-447e-8b5a-14b4317f1265"
6      "progress":
7      {
8          "assetId": 958291
9          "title": "God morgen Norge"
10         "offset":
11         {
12             "duration": 7133249
13             "pos": 7101143
14         }
15         "category":
16         {
17             "id": 919990
18             "name": "Innev?rende sesong"
19         }
20     }
21 }
```

Listing 4: Non silverlight event example

In addition to these information, an event generated by silverlight plugin contains an additional field with details about the client used for playing a particular asset. In listing 5 we reported the most important fields for our analysis.

4.3 Output results

One of the limit of lambda architecture is that the output needs to be defined a priori. According to the considerations made in section 3.2.5, the following output views has been defined:

1. Number of times of assets starting or ending.

```

1  {
2      "client":
3      {
4          "playerEvent": "period"
5          "userAgent": "Mozilla/5.0 (Windows NT 6.1; WOW64; rv:40.0)
6          Gecko/20100101 Firefox/40.0"
7          "env.platform": "silverlight"
8          "buildName": "Vimond.Player.TV2Sumo"
9          "video.format": "smil"
10     }
11 }

```

Listing 5: Silverlight event example

2. Most started assets.
3. Most ended assets.
4. Most browsers used when starting an asset.
5. Most OSs used when starting an asset.
6. Most video formats used when starting an asset.

In order to have consistent outputs across the different batch views we exploit the field *"playerEvents"* which gives information of the type of event performed by the user. It can assume the values *start*, *period* and *end*. We could not use the event marked with *period* as the different batch windows does not share information and then every results produced out of them would have not been consistent. A *playerEvent=start* or *playerEvent=end* represents instread a unique event. Unfortunately, only a subset of the input data contains information about the player event (i.e. the one generated by the silverlight plugin). A request has been forwarded to TV2 for having these information also in the events generated by other devices. Moreover it has been requested to change

the mode of generating *end* events: an user likely stops the streaming of an asset before it actually ends, for instance when skipping the ending titles, and this results in a smaller number of this kind of events. An alternateve solution would be to consider an asset to be ended when the streaming reached a certain percentage over its lenght, for instance 95%. These features are going to be implemented within the next platform release.

4.4 Batch layer

4.4.1 EventFetcher

EventFetcher is the process in charge of reading events from Kafka a store them in the HDFS according to their timestamp. There are two different execution mode: unreliable and realiable. Figure 4.2 shows the class diagram for the main classes involved, reporting the most important fields and methods.

The *UnreliableKafkaConsumerService* extends a kafka consumer class defined in the Vimond repository since the latter offers method for reading messages from a kafka brokers and committing the offset immediately after. What was missing was the ability to process the messages in batches. Each consumer threads reads from kafka and puts the message in a *LinkedBlockingQueue*. The usage of this object is due to its thread-saftness. Each batch operation is triggered either by time or by a threshold on the number of messages stored in memory. Whenever one of this events occurs, the messages are copied into a separate buffer and written in batch into the HDFS from a separate task. This ensure that the storing messagest is not a bottleneck in the overall process, but does not give any guarantees about the correct execution of the operation since the consumer threads are not aware of the writing one.

The *ReliableKafkaConsumerService* instread, behaves similarly to the unreliable coun-

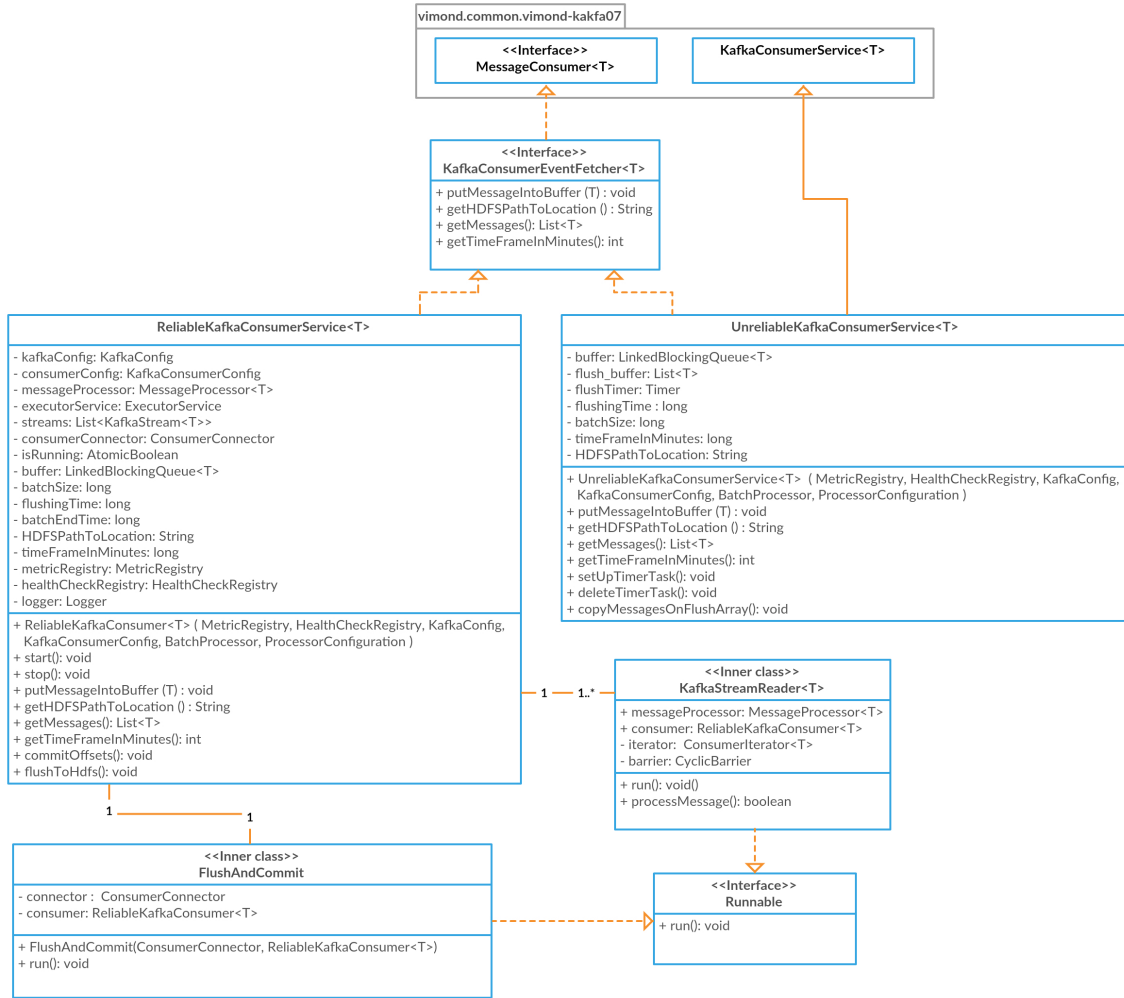


Figure 4.2: EventFetcher class diagram

terpart, but does not commit a message offset after reading it, only when a batch has been successfully written on HDFS. All the consumer thread are synchronized by a *CyclicBarrier* object. Every time a message is received, it is stored in a common buffer and each consumer thread checks if the message threshold or a batch interval has been reached and in case blocks itself on the barrier. The last thread to complete these operations is the one delegated to write the messages into the HDFS: it first tries to contact the HDFS namenode and, in case of its availability, it proceeds with the writing part. In case the namenode is not available, for instance due to a network error, it keeps

retrying to contact it using an exponential backoff policy. When the writing process is completed, the offset is finally committed since that batch of messages is considered processed. In order to avoid deadlock, the other threads wait on the barrier for no more than a configurable time, after that, the overall programs raise an exception and exits. This is not the best approach even though guarantees that the next execution of the program reads the same messages that otherwise could have been lost.

4.4.2 Batch data pipeline

As we said before, we used Apache Spark for implementing this layer. Spark itself does not provide any scheduling mechanism but just a single execution of the aggregation process. Normally an action scheduler is needed for the repetition of the same batch job, each time parametrized in a different way. Considering the usage of Hadoop HDFS as a solution for the data storage, it resulted convenient using Apache Oozie, the official job scheduler over Hadoop. As we explained in chapter 2, Oozie allows to schedule a DAG of actions, where an action is executed only when the previous one ended.

Picture 4.3 shows the sequence of actions performed at every execution:

- start, fail and end are control nodes.
- spark-node and java-node are action nodes.

The user specifies the input parameters such as the beginning time or the frequency of the execution through a property file containing a list of *key=value* tuple. A execution of the DAG is defined through an XML file called *workflow.xml*, while the scheduling is defined through a *coordinator.xml* file. Oozie allows to define parametrized parameter which are resolved with a different value according to the current execution: in this case the job takes a different input folder according to the time of the execution as showed in listing 6.

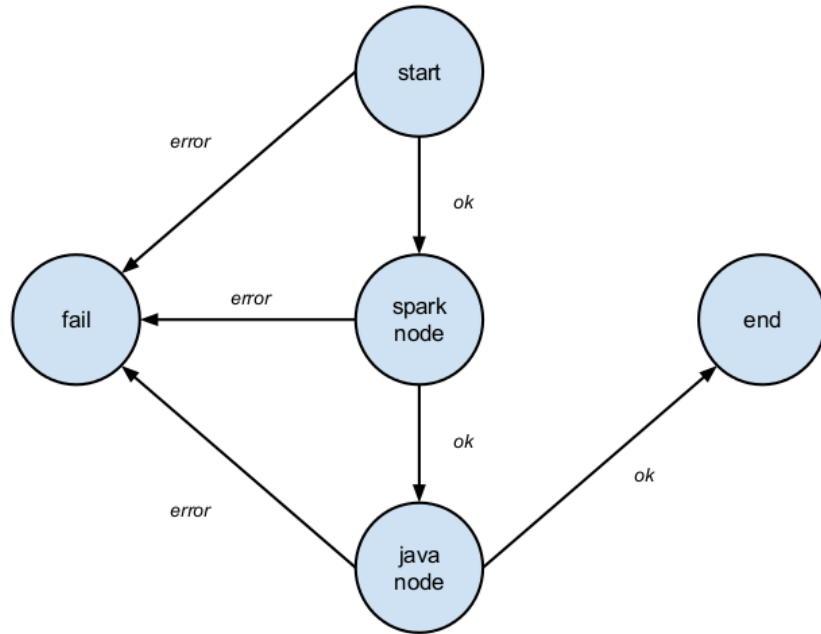


Figure 4.3: Oozie data pipeline

With the *dataset* tag it is possible to define a parametric path where to take the data according to the *YEAR-MONTH-DAY/HOUR/MINUTE* pattern. A dataset can be an input or output one. In this case, with the tag *data-in* we consider an input dataset resolved into the dataset generated two batch window before the current execution. For instance, let us assume to have a 30 minutes batch window and let the next to be executed at 17 pm of a certain day: that job will work on the dataset containing the data from 16:00 to 16:30. This is a complication derived from kafka: since it does not ensure message ordering we cannot be sure that at the time of the execution of a job, all the events of the previous timeframe have been written on the correct folder. We have checked empirically that, delaying the execution of a job of two timewindows guarantees that all the events are considered in the execution.

After the validation of the input parameters, the execution continues through the spark node. This is the step where raw data are actually analyzed and aggregated

```
<datasets>
  <dataset name="vimondEvents" frequency="${coord:minutes(30)}"
    initial-instance="${datasetStartTime}" timezone="GMT+02:00">
    <uri-template>${nameNode}/user/${coord:user()}/${datasetFolder}
      /master/${YEAR}-${MONTH}-${DAY}/${HOUR}/${MINUTE}</uri-template>
    </dataset>
</datasets>
<input-events>
  <data-in name="input" dataset="vimondEvents">
    <instance>${coord:current(-2)} </instance>
  </data-in>
</input-events>
```

Listing 6: RealTime data example

in order to have a compact view on them. Since we already use Hadoop, in order to avoid the deployment of another processing system, we decided to run Spark jobs on Yarn. As stated in chapter 2, Yarn is independent from the computing paradigm as it only allocates resources to containers which make requests for them. There are two modalities of running Spark on Yarn: *client-mode* and *cluster-mode*. In the first one, the Spark driver program is run on the client while in the last one the driver runs within the same container of the Application master.

The program written using Spark library does a strong use of in-memory capability of the framework. The following picture represents the steps executed within the program.

Spark provides for methods for loading binary data directly from HDFS, so no effort is required for converting byte arrays in String. Every loaded event is then converted into a *SparkEvent* object for being analyzed. Since Kafka offers an *at-least-one* message semantic, there might be duplicates, hence an additional filter process is then needed in order to remove them. Unfortunately this is the most expensive step as it has to analyze the whole dataset, and normally shuffle operations between the executors are needed.

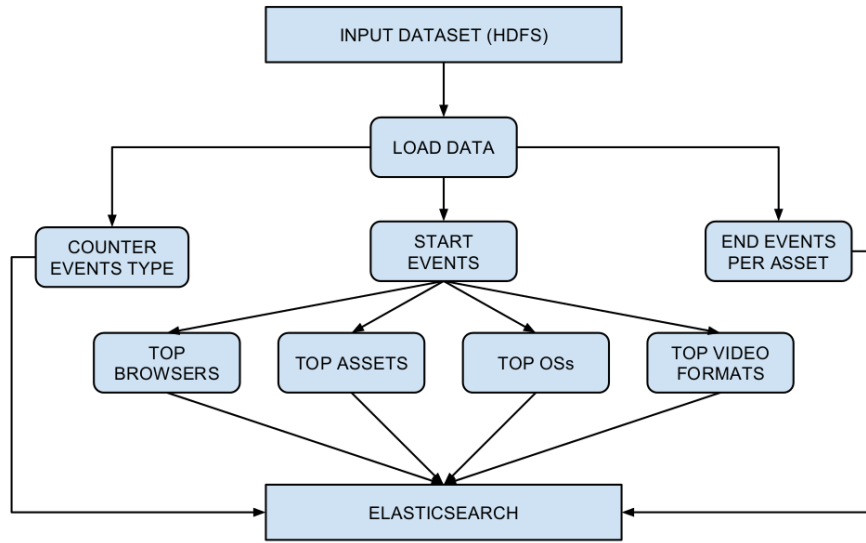


Figure 4.4: Spark actions

The dataset obtained in this step (LOAD DATA block) is then cached in memory since it is needed by every other process yet to be run in the current execution. In case of dataset bigger than the memory available, it is spilled to the disk and deleted when it is no longer needed.

From this dataset, three branches are executed:

- Counter events type: this step calculates the number of start and end events
- End events per asset: this step calculates how the end events are distributed among the assets, i.e. what are the assets that users watch till the end
- Start events: this step is just an additional filter over the loaded dataset. Only player start events are considered. From the output of this process are executed four additional steps which calculate what are the most used browsers, operative systems, video formats and assets.

All the branches eventually write data on the serving layer in form of JSON events.

The aggregation processes are quite simple as they are composed only of map and

reduce operation over the dataset. For completion we report a pseudocode of the process who calculates the number of player-end event per asset.

Algorithm 1: End events per asset aggregation

```

Input: AllEvents input
Result: Aggregated data written on the serving layer
begin
  endEvents  $\leftarrow$  emptyRDD()
  foreach event e in input do
    if e.playerEventType == end then
      endEvents.add(e)
  pairEvents  $\leftarrow$  emptyRDD()
  foreach event e in endEvents do
    tuple  $\leftarrow$  (event.assetName, 1)
    pairEvents.add(tuple)
  reducedEvents  $\leftarrow$  emptyRDD()
  foreach tuple x, y in pairEvents do
    if x.assetName == y.assetName then
      reducedTuple  $\leftarrow$  (x.assetName, x.counter + y.counter)
      reducedEvents.add(reducedTuple)
  outputData  $\leftarrow$  emptyRDD()
  foreach tuple x in reducedEvents do
    output  $\leftarrow$  createBatchData(x)
  writeDataOnServingLayer(outputData)

```

This is just an example of how output data are generated, the process itself does not iterate several times on all the datasets but exploits the distributed power provided by Spark. A dataset is divided in several partition and distributed on all the working processes which execute the pseudocode above illustrated on a smaller sub dataset, fetching data from other partitions only when needed.

Figure 4.5 shows a class diagramm for the most important classes involved in the aggregation process. An aggregation action has been mapped into a Java object implementing the *Job* interface. In addition, for the processes using an input dataset, it has been defined an abstract class named *WorkingJob*. Finally, jobs are started through a *JobStarter* object which uses a factory pattern for jobs creation.

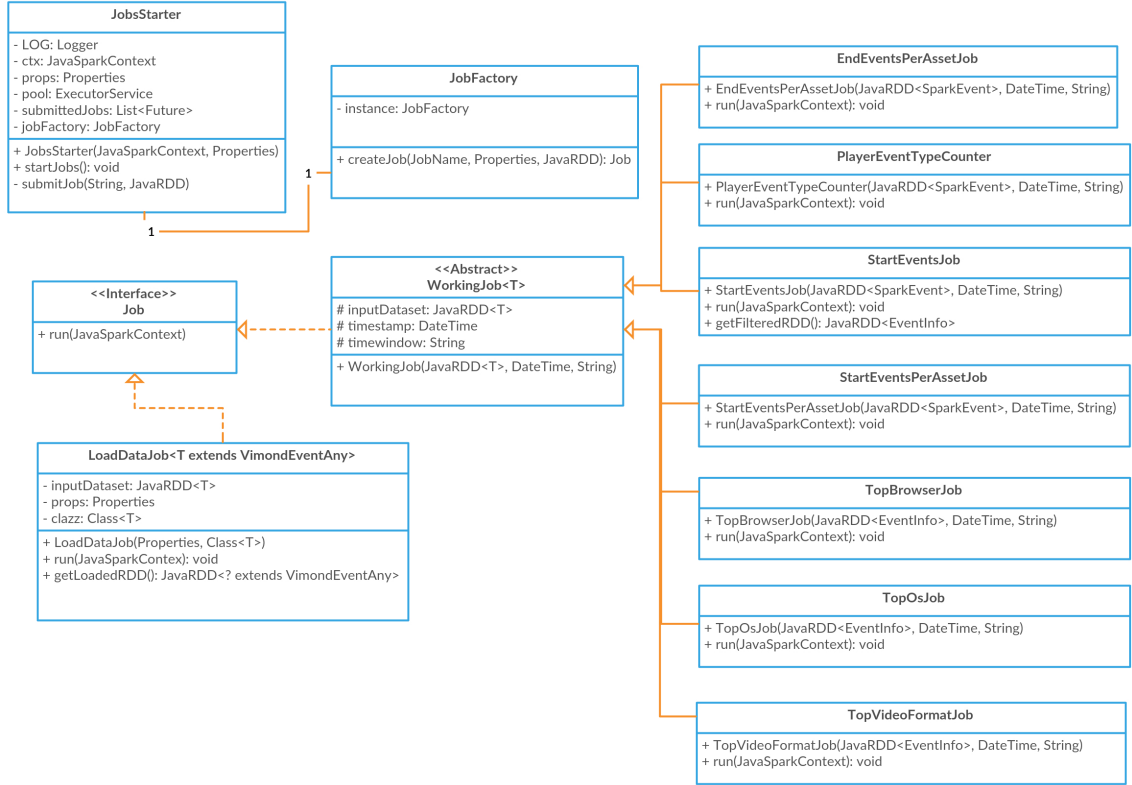


Figure 4.5: Batch class diagram

The last step of the batch pipeline is the deletion of the realtime data after the completion of a batch aggregation process over a timewindow of data. This is simply obtained by querying the the serving layer for the data contained in the current time interval and then delete the corresponding results. The serving layer implemented with elastic-search does not support SQL-like delete operations as *DELETE from table WHERE condition*, but it requires the *id* of the document to be deleted. This process is done in batches in order to improve the throughput of the operations as it would be network expensive sending a delete request for each document. The overall process goes under the name of *delete-by-query*.

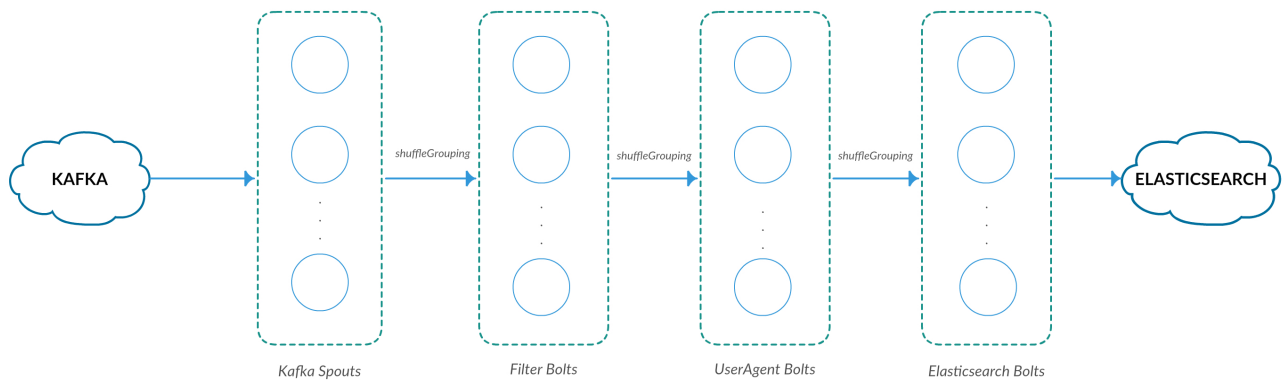


Figure 4.6: Storm real-time topology

4.5 Real time layer

4.6 Serving layer

The serving layer has been implented using Elasticsearch. As stated in chapter 2, Elasticsearch provides storage space for documents and a *Lucene* search engine for querying them. Moreover it works with documents in JSON format, making easy to query and aggretrate them according to a specific field inside them.

Mapping.

Query

Kibana

4.7 Tuning

4.7.1 RealTime

Serialization with kryo rather than objectmapper. show benchmark analysis on serialization libraries

Chapter 5

Results and Tests

5.1 Monitoring the system

In order to monitor the system and get statistics about the usage indices of each layer it has been used the library *dropwizard-metrics*¹ (formerly *codahale metrics*).

5.2 Testing VimondAnalytics

In this section we are going to show the results of the tests made on a working implementation running on AWS. When not mentioned, the settings and the virtual machine configurations are the same of the ones indicated in section...

5.2.1 Tests methodologies

firehose throttled

¹<http://dropwizard.github.io/metrics>

Chapter 6

Conclusions

List of Figures

1.1	Google Analytics dashboard	7
1.2	Lambda-architecture design	12
2.1	Kafka topic	19
2.2	Kafka consumer group	20
2.3	YARN architecture	22
2.4	HDFS architecture	23
2.5	Spark cluster architecture	25
2.6	Storm cluster architecture	27
3.1	Vimond architecture example	31
3.2	VimondAnalytics architecture	32
3.3	Data lake structure	36
4.1	Firehose data bridge	41
4.2	EventFetcher class diagram	45
4.3	Oozie data pipeline	47
4.4	Spark actions	49
4.5	Batch class diagram	51
4.6	Storm real-time topology	52

Bibliography

- [1] Google trends "<http://www.google.com/trends/explore#q=big%20data>' (website)
- [2] Ward J. S., Barker A. "*Undefined by data: A survey of Big Data Definitions*" (paper)
- [3] Laney D. "*3D Data Management: Controlling Data Volume, Velocity, and Variety*" 2001 (report)
- [4] Cooper, A "*A Brief History of Analytics*" JISC CETIS Analytics Series, Vol. 1, No. 9. (report)
- [5] Fourie, I., Bothma, T "*Information seeking: an overview of web tracking and the criteria for tracking software*" Aslib Proceedings 2007 (paper)
- [6] Russom P. "*Big Data Analytics*" TDWI best practices report 2011 (report)
- [7] Glass K., Colbaugh R. "*Web Analytics for Security Informatics*" (paper)
- [8] McKinsley Global Institute "*Big data: The next frontier for innovation, competition, and productivity*" 2011 (report)
- [9] Garber L. "*Using In-Memory Analytics to Quickly Crunch Big Data*" 2012 (report)
- [10] Cugola L., Margara A. "*Processing Flows of Information: From Data Stream to Complex Event Processing*" 2012 (paper)

- [11] Web Analytics Association "*Web Analytics Definitions*" 2008 (report)
- [12] Vimond Media Solution "*<http://www.vimond.com>*" (website)
- [13] Nathan Martz, "*How to beat the cap theorem*" (blog article)
<http://nathanmarz.com/blog/how-to-beat-the-cap-theorem.html>
- [14] BIGTOP-1875, "*<https://issues.apache.org/jira/browse/BIGTOP-1875>*"
- [15] Martz Nathan, Warren James "*Big Data: Principles and Best Practices of Scalable Realtime Data Systems*" Manning, 2015 (book)
- [16] Gilbert Seth, Lynch Nancy: "*Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*" 2002 (paper)
- [17] Heule Stefan, Nunkesser Marc, Hall Alexander: "*HyperLogLog in Practice: Algorithmic Engineering of a State of The Art Cardinality Estimation Algorithm*" Proceedings of the EDBT 2013 Conference, ACM, Genoa, Italy 2013 (paper)
- [18] Boykin O., Ritchie S., O'Connel I., Lin J.: "*Summingbird: A Framework for Integrating Batch and Online MapReduce Computations*" 2013 (paper)
- [19] Zaharia M. et. Al : "*Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*" In Proceedings of the 9th USENIX conference on Networked Systems Design and Implementation 2012 (paper)
- [20] Kreps Jay, "*Questioning the Lambda Architecture*" 2014 (blog article)
<http://radar.oreilly.com/2014/07/questioning-the-lambda-architecture.html>
- [21] Kreps Jay, Narkhede Neha, Rao Jun "*Kafka: a Distributed Messaging System for Log Processing*" 2011 (paper)

BIBLIOGRAPHY

- [22] Dean Jeffrey, Ghemawat Sanjay "*MapReduce: Simplified Data Processing on Large Clusters*" 2008(paper)
- [23] Vavilapalli et Al. "*Apache Hadoop YARN: Yet Another Resource Negotiator*" 2013 (paper)
- [24] HDFS High Availability "*<http://hadoop.apache.org/docs/r2.7.0/hadoop-project-dist/hadoop-hdfs/HDFSHighAvailabilityWithNFS.html>*" (website)
- [25] Apache oozie "*<http://oozie.apache.org/>*" (website)
- [26] Zaharia M. et Al. "*Spark: Cluster Computing with Working Sets*" 2013 (paper)
- [27] Zaharia M. et Al. "*Resilient Distributed Dataset: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*" 2013 (paper)